

A MLflow Integration Example

In the following, we describe an example that uses the example code provided by MLflow Recipes, applying NB2P to extract the pipeline and convert it to an executable MLflow Recipe. The result is deployed on the MLflow server and is tracked by its Python APIs or CLI commands.

A.1 Notebook

Below is an example input notebook solving the NYC taxi fare problem, described as an example of MLflow Recipes⁶. Each listing involves a code cell.

```
import pandas as pd
import logging
import sklearn
from sklearn.model_selection import train_test_split
from pandas import DataFrame
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, StandardScaler
```

```
DATA_LOCATION = "../data/nyc.parquet"
```

```
df = pd.read_parquet(DATA_LOCATION, index_col=0)
```

```
def create_dataset_filter(dataset: pd.DataFrame) -> pd.Series[bool]:
    """
    Mark rows of the split datasets to be additionally filtered. This function will be called on
    the training, validation, and test datasets.
    :param dataset: The {train,validation,test} dataset produced by the data splitting procedure.
    :return: A Series indicating whether each row should be filtered
    """

    return (
        (dataset["fare_amount"] > 0)
        & (dataset["trip_distance"] < 400)
        & (dataset["trip_distance"] > 0)
        & (dataset["fare_amount"] < 1000)
    ) | (~dataset.isna().any(axis=1))
```

```
df_train, df_test = train_test_split(df, test_size=0.2, random_state=42)
df_train, df_val = train_test_split(df, test_size=0.25, random_state=42)
```

```
df_train, df_val, df_test = (
    create_dataset_filter(df_train),
    create_dataset_filter(df_val),
    create_dataset_filter(df_test),
)
```

```
def calculate_features(df: DataFrame):
    df["pickup_dow"] = df["tpep_pickup_datetime"].dt.dayofweek
    df["pickup_hour"] = df["tpep_pickup_datetime"].dt.hour
    trip_duration = df["tpep_dropoff_datetime"] - df["tpep_pickup_datetime"]
    df["trip_duration"] = trip_duration.map(lambda x: x.total_seconds() / 60)
    dateTimeColumns = list(df.select_dtypes(include=["datetime64"]).columns)
    df[dateTimeColumns] = df[dateTimeColumns].astype(str)
    df.drop(columns=["tpep_pickup_datetime", "tpep_dropoff_datetime"], inplace=True)
    return df
```

```
function_transformer_params = (
    {} if sklearn.__version__.startswith("1.0") else {"feature_names_out": "one-to-one"}
)
```

⁶<https://github.com/mlflow/recipes-examples/blob/main/regression/>

```

p = Pipeline(
    steps=[
        (
            "calculate_time_and_duration_features",
            FunctionTransformer(calculate_features, **function_transformer_params),
        ),
        (
            "encoder",
            ColumnTransformer(
                transformers=[
                    (
                        "hour_encoder",
                        OneHotEncoder(categories="auto", sparse=False),
                        ["pickup_hour"],
                    ),
                    (
                        "day_encoder",
                        OneHotEncoder(categories="auto", sparse=False),
                        ["pickup_dow"],
                    ),
                    (
                        "std_scaler",
                        StandardScaler(),
                        ["trip_distance", "trip_duration"],
                    ),
                ],
            ),
        ),
    ],
)

```

```
df_train = p.fit(df_train)
```

```
from sklearn.linear_model import SGDRegressor
```

```
estimator_params = {}
```

```
model = SGDRegressor(random_state=42, **estimator_params)
```

```
model.fit(df_train.drop(columns=["fare_amount"]), df_train["fare_amount"])
```

```

nyc_output = model.predict(df_val.drop(columns=["fare_amount"]))
nyc_output.to_parquet("../data/nyc_output.parquet")

```

A.2 Output of NB2P

After inputting the notebook into NB2P, the system outputs two entities: the pipeline graph structure and the code for each component. These entities can be used to render a pipeline visualization.

Component: comp1

```

DATA_LOCATION = "../data/nyc.parquet"
df = pd.read_parquet(DATA_LOCATION, index_col=0)

def create_dataset_filter(dataset: pd.DataFrame) -> pd.Series[bool]:
    return (
        (dataset["fare_amount"] > 0)
        & (dataset["trip_distance"] < 400)
        & (dataset["trip_distance"] > 0)
        & (dataset["fare_amount"] < 1000)
    )

```

```

) | (~dataset.isna().any(axis=1))

df_train, df_test = train_test_split(df, test_size=0.2, random_state=42)
df_train, df_val = train_test_split(df, test_size=0.25, random_state=42)
df_train, df_val, df_test = (
    create_dataset_filter(df_train),
    create_dataset_filter(df_val),
    create_dataset_filter(df_test),
)

```

Component: comp2

```

def calculate_features(df: DataFrame):
    df["pickup_dow"] = df["tpep_pickup_datetime"].dt.dayofweek
    df["pickup_hour"] = df["tpep_pickup_datetime"].dt.hour
    trip_duration = df["tpep_dropoff_datetime"] - df["tpep_pickup_datetime"]
    df["trip_duration"] = trip_duration.map(lambda x: x.total_seconds() / 60)
    dateTimeColumns = list(df.select_dtypes(include=["datetime64"]).columns)
    df[dateTimeColumns] = df[dateTimeColumns].astype(str)
    df.drop(columns=["tpep_pickup_datetime", "tpep_dropoff_datetime"], inplace=True)
    return df

function_transformer_params = (
    {} if sklearn.__version__.startswith("1.0") else {"feature_names_out": "one-to-one"}
)
p = Pipeline(
    steps=[
        (
            "calculate_time_and_duration_features",
            FunctionTransformer(calculate_features, **function_transformer_params),
        ),
        (
            "encoder",
            ColumnTransformer(
                transformers=[
                    (
                        "hour_encoder",
                        OneHotEncoder(categories="auto", sparse=False),
                        ["pickup_hour"],
                    ),
                    (
                        "day_encoder",
                        OneHotEncoder(categories="auto", sparse=False),
                        ["pickup_dow"],
                    ),
                    (
                        "std_scaler",
                        StandardScaler(),
                        ["trip_distance", "trip_duration"],
                    ),
                ],
            ),
        ),
    ],
)
df_train = p.fit(df_train)

```

Component: comp3

```

from sklearn.linear_model import SGDRegressor

```

```
estimator_params = {}
model = SGDRegressor(random_state=42, **estimator_params)
model.fit(df_train.drop(columns=["fare_amount"]), df_train["fare_amount"])
nyc_output = model.predict(df_val.drop(columns=["fare_amount"]))
nyc_output.to_parquet("../data/nyc_output.parquet")
```

Pipeline Graph Structure

The pipeline graph is a JSON file with information about the vertices and edges, based on the naming of the above components.

```
{
  "vertices": [
    "comp1",
    "comp2",
    "comp3"
  ],
  "edges": [
    {"source": "comp1", "target": "comp2", "flow": ["df"]},
    {"source": "comp1", "target": "comp3", "flow": ["df_val"]},
    {"source": "comp2", "target": "comp3", "flow": ["df_train"]}
  ]
}
```

A.3 Converted MLflow Recipe

For MLflow Recipe conversion, we use a template and fill in the necessary information extracted from NB2P. For example, the steps directory contains corresponding code from each component, with template function definitions.

recipe.yaml

```
recipe: "regression/v1"
target_col: "fare_amount"
primary_metric: "root_mean_squared_error"
steps:
  ingest:
    using: parquet
    location: "../data/nyc.parquet"
  transform:
    using: custom
    transformer_method: comp2
  train:
    using: custom
    estimator_method: comp3
  evaluate:
    validation_criteria:
      - metric: root_mean_squared_error
        threshold: 10
      - metric: mean_absolute_error
        threshold: 50
  register:
    allow_non_validated_model: false
  ingest_scoring:
    using: parquet
    location: "../data/nyc.parquet"
  predict:
    output:
      using: parquet
      location: "../data/nyc_output.parquet"
```

profiles/local.yaml

```
experiment:
  name: "taxi_fare_regression"
  tracking_uri: "sqlite:///metadata/mlflow/mlruns.db"
```

```
artifact_location: "./metadata/mlflow/mlartifacts"
model_registry:
  model_name: "taxi_fare_regression"
```

steps/ingest.py

```
import logging

import pandas as pd
import sklearn
from pandas import DataFrame
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, StandardScaler

def create_dataset_filter(dataset: pd.DataFrame) -> pd.Series[bool]:
    return (
        (dataset["fare_amount"] > 0)
        & (dataset["trip_distance"] < 400)
        & (dataset["trip_distance"] > 0)
        & (dataset["fare_amount"] < 1000)
    ) | (~dataset.isna().any(axis=1))

def comp1(file_path: str):
    df = pd.read_parquet(file_path, index_col=0)

    df_train, df_test = train_test_split(df, test_size=0.2, random_state=42)
    df_train, df_val = train_test_split(df, test_size=0.25, random_state=42)
    df_train, df_val, df_test = (
        create_dataset_filter(df_train),
        create_dataset_filter(df_val),
        create_dataset_filter(df_test),
    )

    OUTPUT = df_train, df_val, df_test
    return OUTPUT
```

steps/transform.py

```
import logging

import pandas as pd
import sklearn
from pandas import DataFrame
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, StandardScaler

def calculate_features(df: DataFrame):
    df["pickup_dow"] = df["tpep_pickup_datetime"].dt.dayofweek
    df["pickup_hour"] = df["tpep_pickup_datetime"].dt.hour
    trip_duration = df["tpep_dropoff_datetime"] - df["tpep_pickup_datetime"]
    df["trip_duration"] = trip_duration.map(lambda x: x.total_seconds() / 60)
    dateTimeColumns = list(df.select_dtypes(include=["datetime64"]).columns)
    df[dateTimeColumns] = df[dateTimeColumns].astype(str)
    df.drop(columns=["tpep_pickup_datetime", "tpep_dropoff_datetime"], inplace=True)
    return df
```

```

def comp2():
    function_transformer_params = (
        {}
        if sklearn.__version__.startswith("1.0")
        else {"feature_names_out": "one-to-one"}
    )

    OUTPUT = Pipeline(
        steps=[
            (
                "calculate_time_and_duration_features",
                FunctionTransformer(calculate_features, **function_transformer_params),
            ),
            (
                "encoder",
                ColumnTransformer(
                    transformers=[
                        (
                            "hour_encoder",
                            OneHotEncoder(categories="auto", sparse=False),
                            ["pickup_hour"],
                        ),
                        (
                            "day_encoder",
                            OneHotEncoder(categories="auto", sparse=False),
                            ["pickup_dow"],
                        ),
                        (
                            "std_scaler",
                            StandardScaler(),
                            ["trip_distance", "trip_duration"],
                        ),
                    ]
                ),
            ),
        ]
    )
    return OUTPUT

```

steps/train.py

```

import logging

import pandas as pd
import sklearn
from pandas import DataFrame
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import SGDRegressor
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, StandardScaler

def comp3():
    estimator_params = {}

    OUTPUT = SGDRegressor(random_state=42, **estimator_params)
    return OUTPUT

```

run.py

```

from mlflow.recipes import Recipe

```

```

r = Recipe(profile="local")
r.clean()
r.inspect()

r.run("ingest")
r.run("transform")
r.run("train")
r.run("evaluate")
r.run("register")

r.inspect("train")

training_data = r.get_artifact("training_data")
training_data.describe()

trained_model = r.get_artifact("model")
print(trained_model)

```

A.4 Execution Tracks and Results

The below shows execution tracks and reports from the MLflow server.

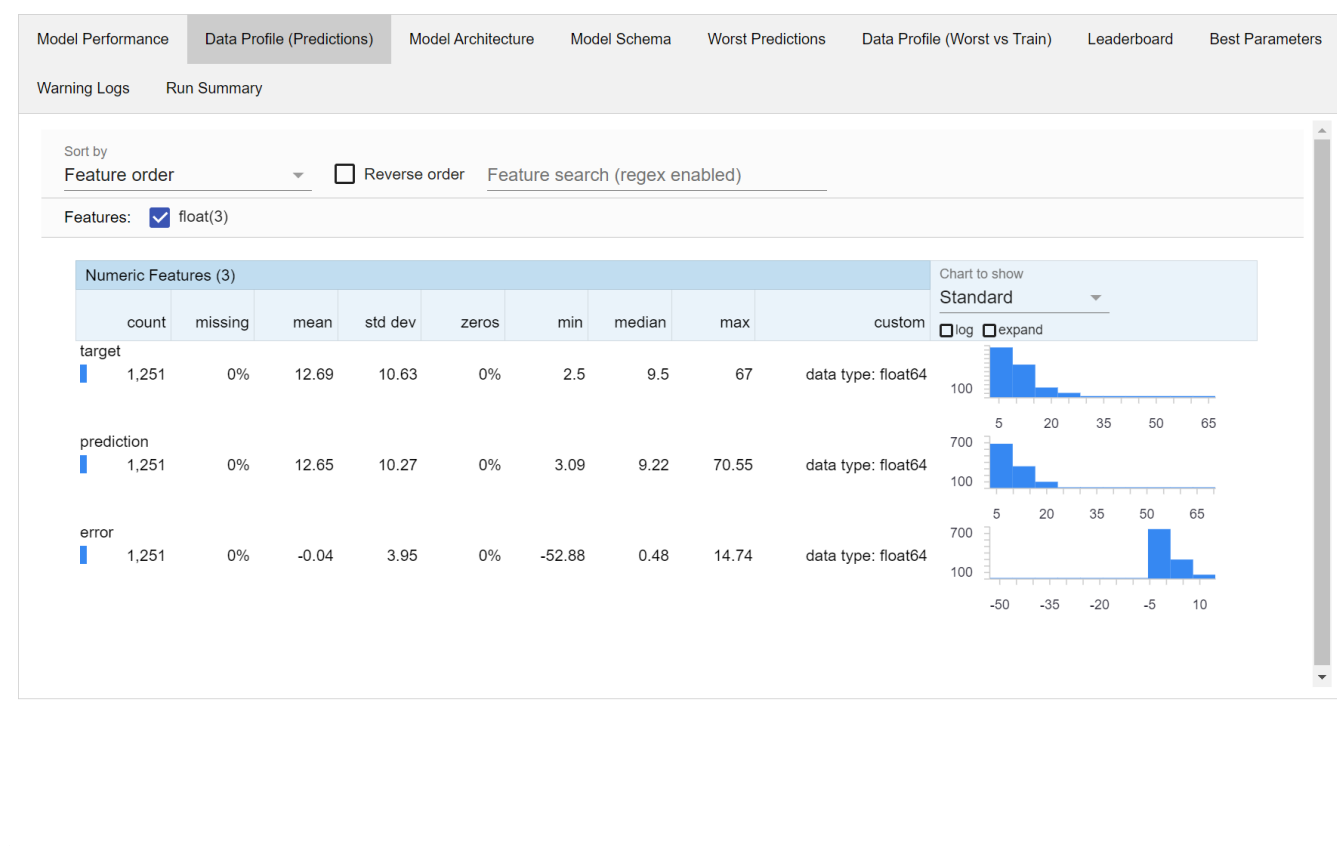


Figure 10

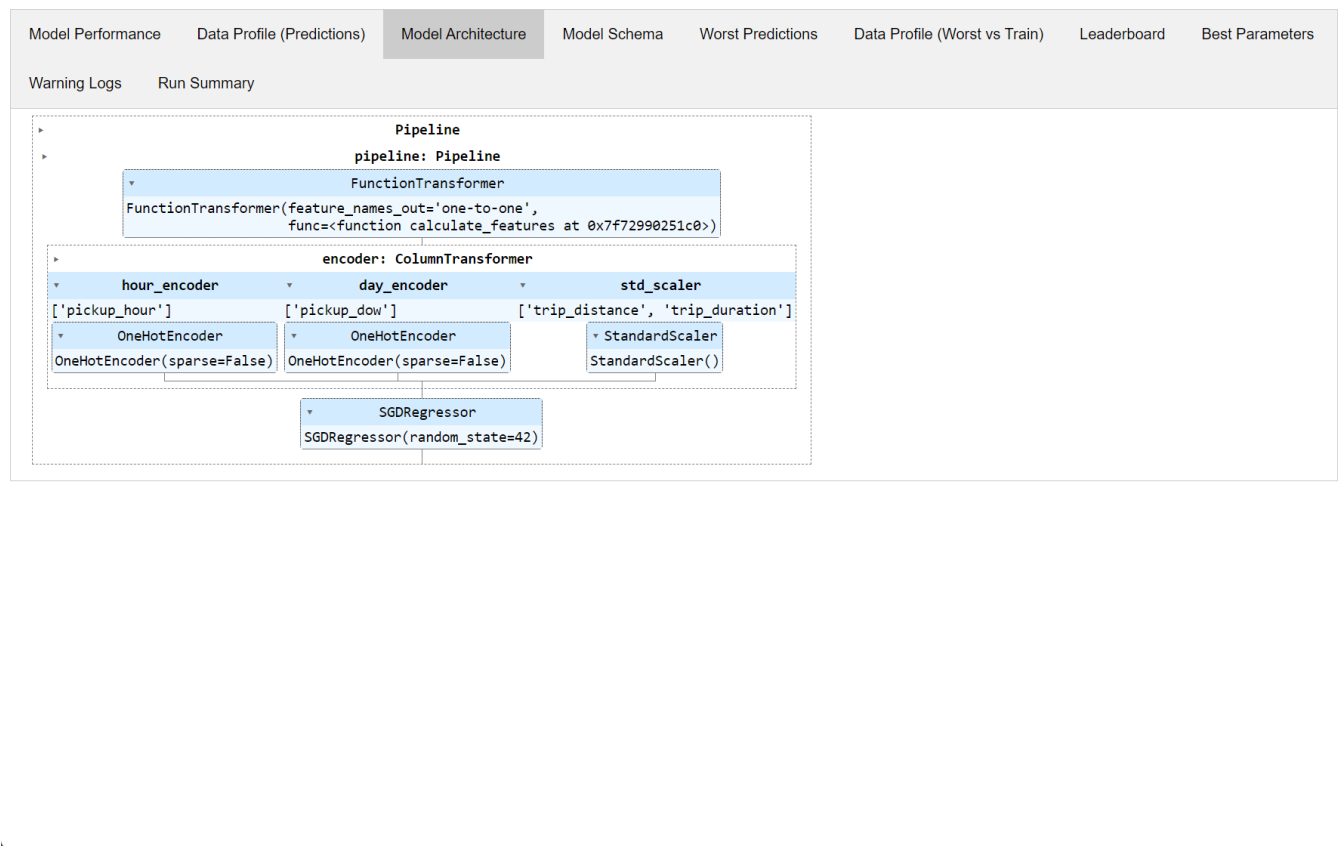


Figure 11

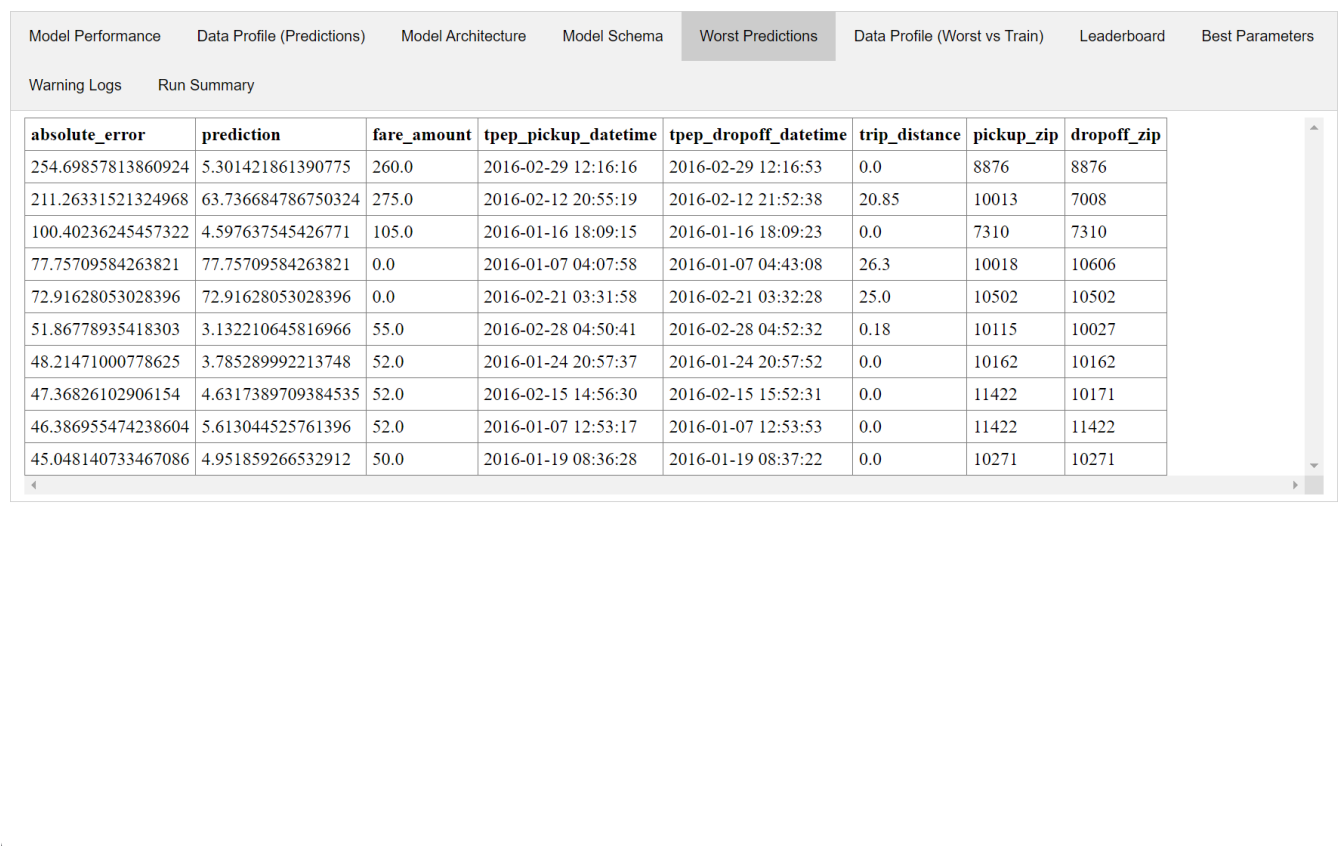


Figure 12

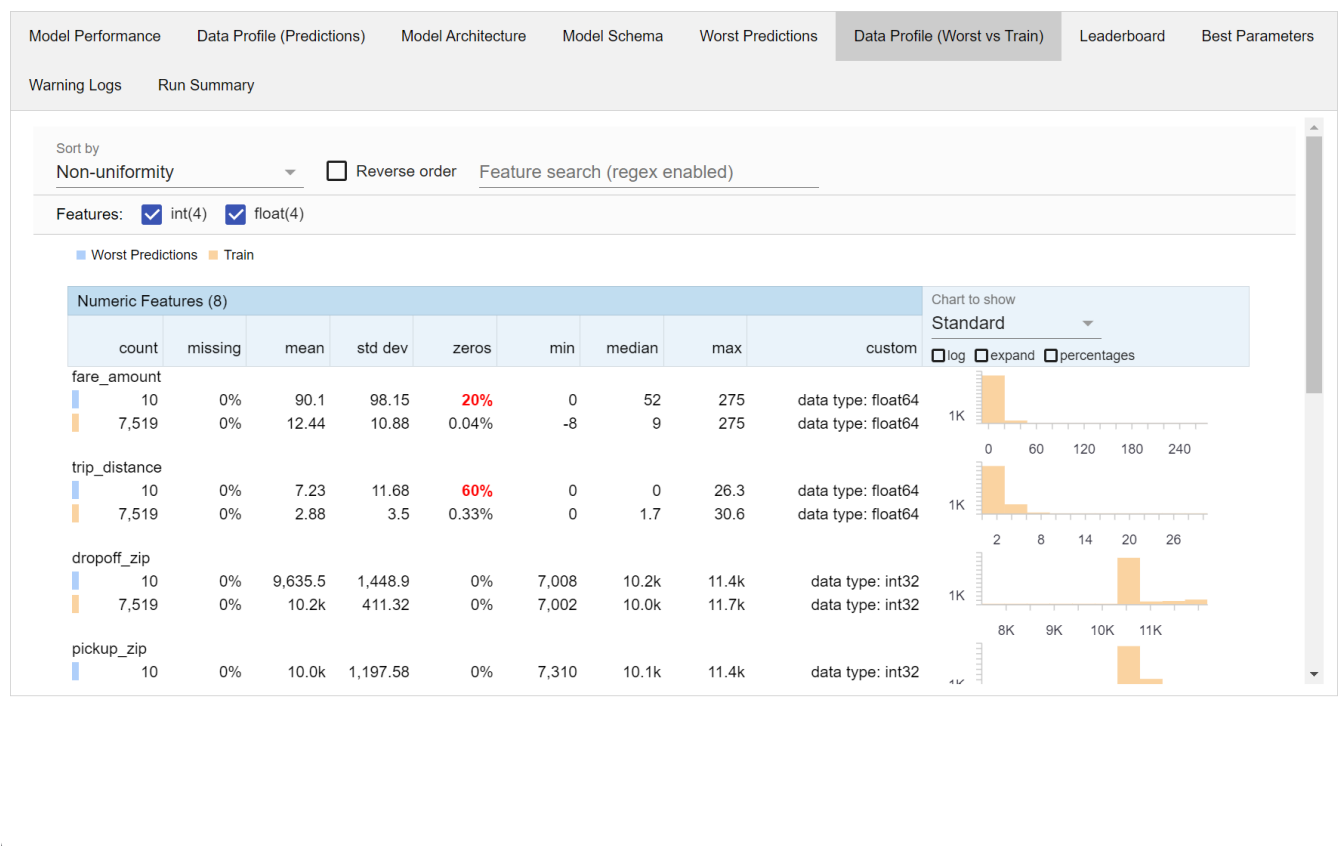


Figure 13

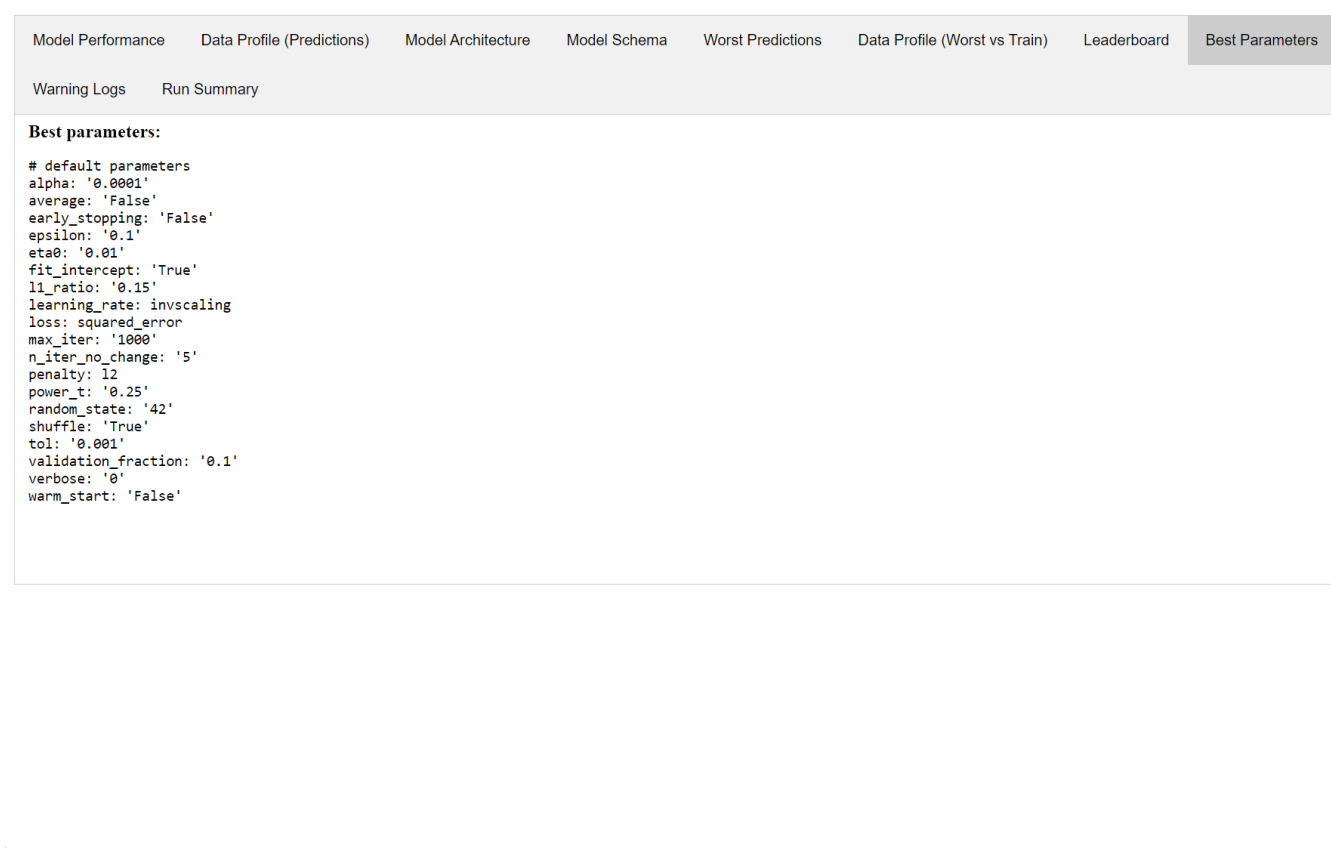


Figure 14

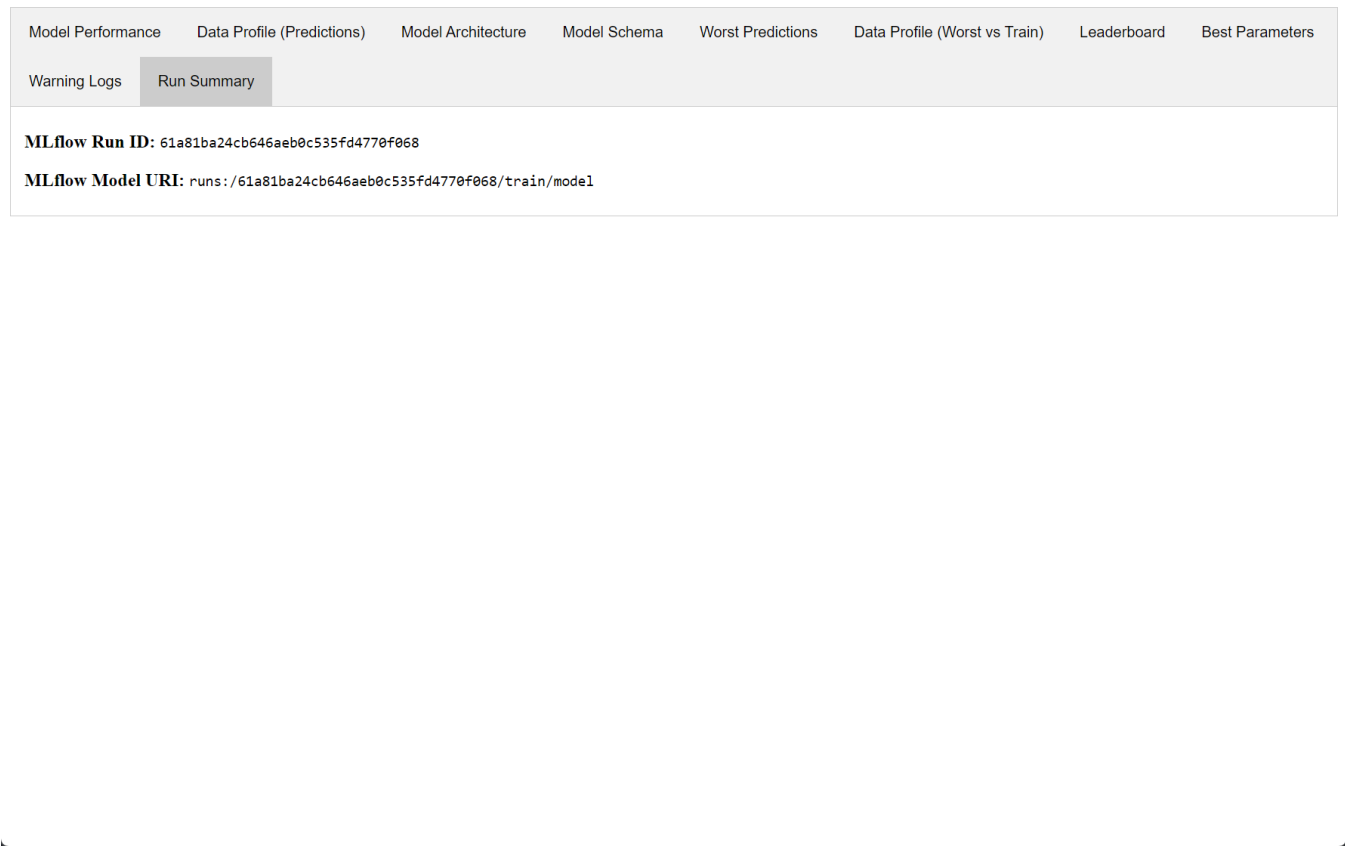


Figure 15